

Unit 6

Unit 6: AWS Cloud & Deployment — Practical/Oral Exam Notes

Context

Part of [Web Application Development](#) | Practical + Oral Exam Prep | SPPU Third Year IT

High-Yield Oral Topics

- **IaaS vs PaaS vs SaaS** — With AWS examples (always asked)
- **EC2** — What it is, how to deploy on it
- **Elastic Beanstalk** — Easiest deployment (PaaS)
- **VPC** — What is it, public vs private subnet
- **ELB Types** — ALB vs NLB
- **S3** — What it stores, how to host static site

MINI PROJECT ANCHOR: Deploy SMS on AWS

Deployment Goal: Host our Node.js + MongoDB SMS backend on AWS EC2 / Elastic Beanstalk.

SECTION 1 — CLOUD COMPUTING BASICS

Exam Definition

Cloud computing is the **on-demand delivery of IT resources** (compute, storage, databases, networking) over the internet with **pay-as-you-go pricing**, eliminating the need to buy/maintain physical hardware.

Cloud Service Models — MOST ASKED

Model	Full Name	You Manage	AWS Service	Example
IaaS	Infrastructure as a Service	OS, Runtime, App, Data	EC2, VPC, EBS	You install Node.js on a server
PaaS	Platform as a Service	App code + Data only	Elastic Beanstalk, RDS	Just upload your ZIP file
SaaS	Software as a Service	Nothing (just use it)	Gmail, Salesforce	End user apps

Memory Trick: IaaS = I manage everything. PaaS = Platform manages infra. SaaS = Someone else manages everything.

Benefits of Cloud (6 Points for Viva)

1. **Pay-as-you-go** — No upfront hardware cost
2. **Elastic Scaling** — Scale up/down instantly with traffic
3. **High Availability** — Multiple data centers, no single point of failure
4. **Global Reach** — Deploy in 20+ regions worldwide in minutes
5. **Managed Services** — AWS handles hardware, power, cooling
6. **Speed & Agility** — Launch a server in 2 minutes vs weeks for physical

SECTION 2 — AWS EC2

Exam Definition

Amazon EC2 (Elastic Compute Cloud) is an **laaS** service that provides **resizable virtual servers (instances)** in the cloud. You have full control over the OS and software stack.

Key EC2 Concepts

Concept	What It Is	SMS Example
AMI	Blueprint for the instance (OS + preinstalled software)	Ubuntu 22.04 Server
Instance Type	Size (CPU, RAM, Storage)	t2.micro (free tier — 1 vCPU, 1GB RAM)
Security Group	Virtual firewall — controls inbound/outbound traffic	Allow port 80, 443, 3000, 22
Key Pair	SSH public/private keys for secure login	sms-key.pem
Elastic IP	Static public IP that doesn't change on restart	54.XX.XX.XX

Step-by-Step: Deploy SMS on EC2

```
# ===== STEP 1: Launch EC2 Instance =====
# 1. Go to AWS Console → EC2 → Launch Instance
# 2. Choose: Ubuntu Server 22.04 LTS (Free Tier)
# 3. Instance type: t2.micro
# 4. Create new Key Pair → Download sms-key.pem
# 5. Security Group: Allow SSH(22), HTTP(80), Custom TCP(3000)
# 6. Launch!

# ===== STEP 2: Connect via SSH =====
# On Windows: Use PuTTY or Windows Terminal

# Convert .pem to .ppk using PuTTYgen (Windows only)
# Then connect via PuTTY: Host = public-ip, Auth = .ppk file
```

```
# On Mac/Linux:
chmod 400 sms-key.pem
ssh -i "sms-key.pem" ubuntu@<your-ec2-public-ip>

# ===== STEP 3: Install Node.js on EC2 =====
sudo apt update
sudo apt upgrade -y

# Install Node.js (via NVM – recommended)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
source ~/.bashrc
nvm install 18          # Install Node.js 18
node --version          # Verify

# Install MongoDB
sudo apt install -y gnupg
# (follow MongoDB Ubuntu install guide)
sudo systemctl start mongod
sudo systemctl enable mongod

# ===== STEP 4: Deploy SMS Backend =====
# Option A: Clone from GitHub
git clone https://github.com/yourusername/sms-backend.git
cd sms-backend
npm install

# Create .env file
nano .env
# Add: MONGO_URI=mongodb://localhost:27017/smsdb
# Add: PORT=3000

# ===== STEP 5: Run with PM2 (Process Manager) =====
npm install -g pm2

pm2 start server.js --name "sms-api"
pm2 startup          # Auto-restart on server reboot
pm2 save
pm2 logs sms-api     # View logs
pm2 status           # Check running processes

# ===== STEP 6: Access Your App =====
# http://<ec2-public-ip>:3000/api/students
# Or set up Nginx as reverse proxy on port 80
```

PM2 — Microservices Tool (Assignment 3)

PM2 is a **Process Manager** for Node.js applications. It keeps your app **always running** (restarts on crash), supports **clustering** (run on multiple CPU cores), and provides real-time **monitoring**. This enables **microservices** — running multiple independent services.

```
# Run multiple services (microservices)
pm2 start student-service.js --name "student-service" -- --port 3001
pm2 start auth-service.js    --name "auth-service"    -- --port 3002
pm2 start report-service.js  --name "report-service"  -- --port 3003

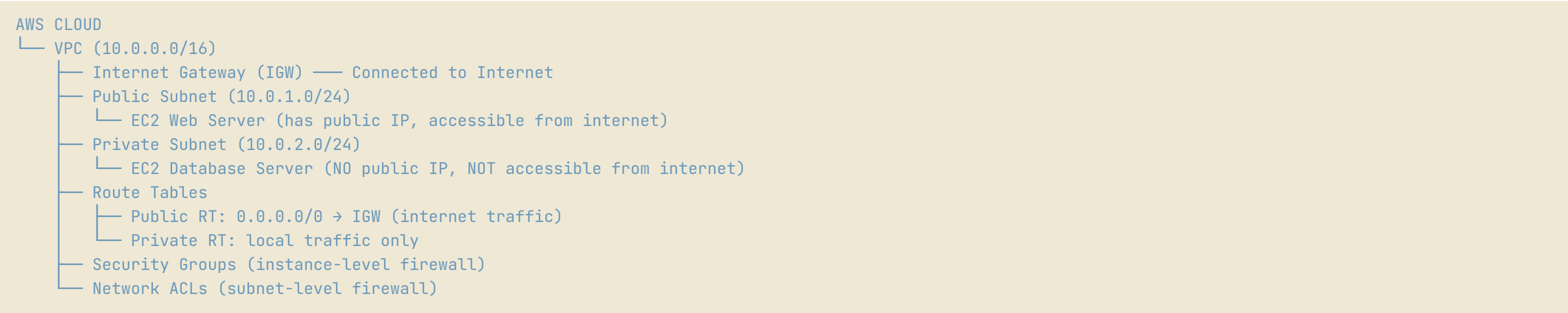
pm2 list      # See all services
pm2 monit     # Real-time monitoring dashboard
```

SECTION 3 — AWS VPC

Exam Definition

VPC (Virtual Private Cloud) is a **logically isolated section of the AWS cloud** where you can launch AWS resources in a virtual network that you define. It gives you complete control over your networking environment.

VPC Components



VPC Key Concepts Table

Component	Role	SMS Example
VPC	Isolated virtual network	10.0.0.0/16 CIDR block
Subnet	Sub-division of VPC	Public (web), Private (DB)
Internet Gateway	VPC-to-Internet door	Allows EC2 to be web-accessible

Component	Role	SMS Example
Route Table	Network traffic map	Send internet traffic via IGW
Security Group	Instance firewall	Allow port 3000 inbound
NAT Gateway	Private subnet → internet	DB server downloads updates

SECTION 4 — ELASTIC LOAD BALANCER (ELB)

📌 Exam Definition

ELB automatically **distributes incoming traffic across multiple EC2 instances** to ensure high availability, fault tolerance, and optimal performance.

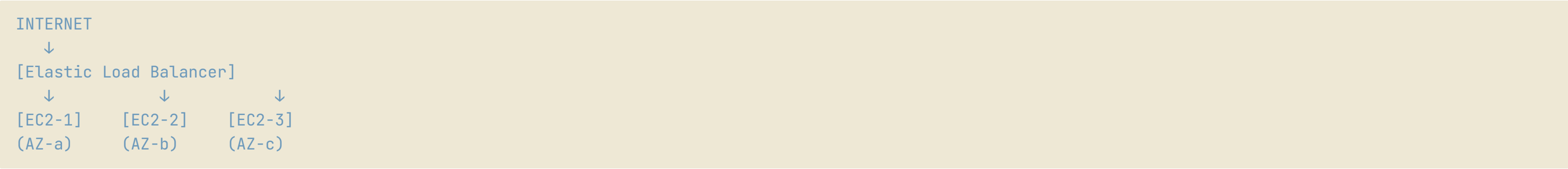
ELB Types — MUST KNOW

Type	Layer	Smart Routing?	Use Case
ALB (Application LB)	Layer 7 (HTTP/S)	Yes — URL path/host based	Web apps, microservices
NLB (Network LB)	Layer 4 (TCP/UDP)	No — IP/port based	High performance, gaming
GWLB (Gateway LB)	Layer 3	No	Third-party firewalls
CLB (Classic LB)	Layer 4/7	Basic	Legacy (avoid)

ALB Smart Routing Example:

- Traffic to `/api/students` → Routes to Student Service EC2 instances
- Traffic to `/api/auth` → Routes to Auth Service EC2 instances
- Traffic to `/` → Routes to Frontend EC2 instances

High Availability with ELB



If EC2-1 in Availability Zone A goes down, ELB automatically routes to EC2-2 and EC2-3. **Zero downtime!**

SECTION 5 — AWS STORAGE

S3 vs EBS

Feature	S3 (Simple Storage)	EBS (Elastic Block Store)
Type	Object storage	Block storage
Access	Via URL / API	Mounted to EC2 like HDD
Use Case	Images, videos, backups, static site	EC2 OS disk, databases
Persistence	Indefinite	Lives with EC2
Sharing	Publicly shareable via URL	One EC2 at a time

Host Static Website on S3

```
# 1. Create S3 bucket
# AWS Console → S3 → Create bucket → "sms-frontend-bucket"

# 2. Upload files (Angular build output)
ng build --prod      # Creates dist/ folder
# Upload all files from dist/ to S3 bucket

# 3. Enable Static Website Hosting
# Bucket → Properties → Static website hosting → Enable
# Index document: index.html

# 4. Make public
# Bucket Policy to allow public read access

# Your website is now at:
# http://sms-frontend-bucket.s3-website.ap-south-1.amazonaws.com
```

SECTION 6 — ELASTIC BEANSTALK (PaaS)

Exam Definition

AWS Elastic Beanstalk is a **PaaS service** that automatically handles deployment, scaling, load balancing, and monitoring. You only upload your **application code** — Beanstalk manages everything else.

What Beanstalk Manages Automatically

What You Provide	What Beanstalk Handles
Application code (ZIP/Docker)	EC2 instances provisioning

What You Provide	What Beanstalk Handles
Environment variables	Load balancer setup
—	Auto scaling configuration
—	Health monitoring
—	OS patches and updates

Deploy SMS on Elastic Beanstalk

```
# ===== METHOD 1: AWS CLI =====
# Install EB CLI
pip install awsebcli

# Initialize Beanstalk in your project
eb init
# Select: ap-south-1 (Mumbai)
# Platform: Node.js
# Application name: sms-backend

# Create environment and deploy
eb create sms-production
eb deploy

# Check status
eb status
eb health
eb logs

# Open the deployed app
eb open

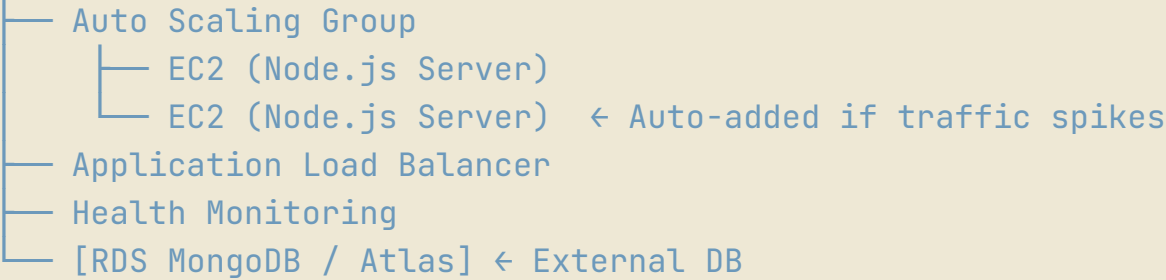
# ===== METHOD 2: AWS Console =====
# 1. Zip your Node.js project: sms-backend.zip
# 2. Console → Elastic Beanstalk → Create Application
# 3. Platform: Node.js
# 4. Upload code → sms-backend.zip
# 5. Configure environment variables (MONGO_URI, PORT)
# 6. Launch → Wait ~5 mins
# 7. Access via: http://sms-backend.ap-south-1.elasticbeanstalk.com
```

Beanstalk Architecture

INTERNET



[Elastic Beanstalk Environment]



SECTION 7 — GIT + GITHUB (Assignment 2)

Git Commands — Must Know for Viva

```
# Initialize Git repository
git init

# Configure identity
git config --global user.name "Rahul Sharma"
git config --global user.email "rahul@example.com"

# Check status
git status

# Stage files
git add .                # Add all files
git add server.js        # Add specific file

# Commit
git commit -m "Initial: SMS backend with CRUD API"

# Connect to GitHub
git remote add origin https://github.com/username/sms-backend.git

# Push to GitHub
git push -u origin main

# Pull latest changes
git pull origin main

# Create & switch branch
```



```
git checkout -b feature/student-login
```

```
# Merge branch
```

```
git checkout main
```

```
git merge feature/student-login
```

```
# View history
```

```
git log --oneline
```

Git Workflow for Deployment

```
LOCAL DEVELOPMENT
```

```
git add . && git commit → git push → GitHub
```

↓

```
AWS CodeDeploy / GitHub Actions
```

↓

```
EC2 auto-pulls and restarts
```

VIVA Q&A — Unit 6

Q1: What is the difference between IaaS, PaaS, and SaaS?

IaaS (EC2): You get a virtual server — install OS, Node.js, your app yourself. Full control. **PaaS** (Elastic Beanstalk): You only provide code — AWS manages servers, scaling, load balancing. **SaaS**: You just use the software as a product (Gmail, Google Docs). As you go from IaaS → SaaS, you manage less.

Q2: What is EC2? How would you deploy a Node.js app on it?

EC2 is AWS's virtual server service. To deploy Node.js: Launch instance (Ubuntu), SSH in with key pair, install Node.js, clone code from GitHub, install dependencies with `npm install`, run with PM2 for persistence, open port 3000 in Security Group.

Q3: What is VPC? What are public and private subnets?

VPC is a logically isolated virtual network in AWS. A **public subnet** has an Internet Gateway route — resources here have public IPs (web servers). A **private subnet** has no internet route — resources here (databases) are not directly accessible from internet, adding security.

Q4: What is Elastic Beanstalk? Why use it over EC2?

Elastic Beanstalk is PaaS — you upload code and it handles EC2, load balancing, scaling, and health monitoring automatically. Use it when you want to **focus on code** without managing infrastructure. Use EC2 when you need **full control** over the server environment.

Q5: What is the difference between ALB and NLB?

ALB (Application Load Balancer) works at Layer 7 (HTTP/HTTPS) and can route based on URL paths, headers, hostnames — great for microservices. **NLB** (Network Load Balancer) works at Layer 4 (TCP/UDP) with ultra-low latency — great for high-performance, non-HTTP applications like gaming or streaming.

Q6: What is S3? How do you host a static website on it?

S3 is object storage for files like images, videos, and HTML. To host a static site: Create bucket, upload HTML/CSS/JS files, enable "Static Website Hosting" in settings, set bucket policy for public read access. The site is then accessible at the S3 bucket URL.

 Oral Trap Questions

Trap: "What is the difference between Security Group and NACL?"

Security Group: Instance-level firewall, **stateful** (if you allow inbound, response is automatically allowed outbound). **NACL** (Network Access Control List): Subnet-level firewall, **stateless** (must explicitly set both inbound and outbound rules). Security Groups are more commonly used.

Trap: "What is Auto Scaling? How does it relate to ELB?"

Auto Scaling automatically adds/removes EC2 instances based on traffic load. ELB distributes traffic across all active instances. Together: When traffic spikes, Auto Scaling adds a new instance → ELB detects it and starts routing traffic to it. When traffic drops, the extra instance is terminated to save cost.

Trap: "What is a Region vs Availability Zone in AWS?"

A **Region** is a geographic area (e.g., ap-south-1 = Mumbai). Each Region has multiple **Availability Zones** (AZs) — isolated physical data centers. Deploying across multiple AZs protects against a single data center failure.

 Assignment 4 — Deployment Viva Flow

"For Assignment 4, I deployed our Student Management System on AWS. I launched an EC2 instance with Ubuntu, connected via SSH, installed Node.js and MongoDB, cloned the project from GitHub, and ran it using PM2 for persistence. I configured the Security Group to allow HTTP on port 80 and our app on port 3000. For easier deployment, I also explored Elastic Beanstalk where I just uploaded the ZIP file and it automatically provisioned all infrastructure."

 High Probability Questions

1. IaaS vs PaaS vs SaaS with AWS examples
 2. What is EC2? Explain AMI and Security Group
 3. Steps to deploy a Node.js app on EC2
 4. What is VPC? Public vs private subnet
 5. What is ELB? Types of ELB?
 6. What is S3? How to host static website?
 7. What is Elastic Beanstalk? Advantages over EC2?
 8. What is PM2? Why use it?
 9. Git commands: init, add, commit, push
 10. What is Auto Scaling?
-

 Quick Revision Table

Service	Type	Key Point
EC2	IaaS	Virtual server — full OS control
Elastic Beanstalk	PaaS	Upload code — AWS manages rest
S3	Storage	Object store — files, static sites
EBS	Storage	Block store — EC2 hard drive
VPC	Networking	Isolated virtual network
ELB	Networking	Traffic distributor across instances
IAM	Security	User/role access control
Route 53	DNS	Domain name → IP mapping
CloudFront	CDN	Global content delivery
RDS	Database	Managed relational DB (MySQL etc.)